

学習記録アプリ 修正内容の説明

対象ファイル: `src/App.tsx` / 使用技術: React・React Hook Form・Chakra UI v3・Supabase / 2026年10月7日

目次

1. 登録が成功したときだけモーダルを閉じる
2. エラー 23502 (title に null が入る) の原因と修正
3. コンソールの [vite] ログと、データ取得が2回走る理由
4. 編集モーダルでの更新機能 (仕様と実装)
5. 残っている課題
6. 参考サイト

1. 登録が成功したときだけモーダルを閉じる

やりたいこと

新規登録で、Supabase への保存が成功したときだけ `setIsOpen(false)` でモーダルを閉じたい。

問題点: Supabase は失敗しても例外を投げない

Supabase の `insert` / `update` / `delete` は、失敗しても **例外 (throw) を発生させません**。代わりに、戻り値の `{ data, error }` の `error` にエラー内容が入ります。

そのため `try...catch` だけでは失敗を検知できず、`catch` を素通りして、失敗してもモーダルが閉じていました。

修正前

```
try {
  await supabase.from('study-record').insert([
    { title: data.title, time: data.time }
  ]);
} catch (error) {
  // ← DBエラーではここに来ない
  console.error("登録エラー", error);
  return;
}
await setIsOpen(false); // ← 失敗しても実行される
fetchData();
```

修正後

```
try {
  const { error } = await supabase.from('study-record').insert([
    { title: data.title,
      time: data.time,
    }
  ]);
  if (error) { // ← 戻り値の error で失敗を判定
    console.error("登録エラー", error);
    return; // ← 失敗時はモーダルを開いたまま
  }
  setIsOpen(false); // ← 成功時のみ閉じる
  await fetchData();
} catch (error) {
  console.error("通信エラー", error); // ← ネットワーク切断など予期しない例外用
}
```

ポイント

- Supabase の結果は `const { error } = await ...` で受け取り、`if (error)` で判定する。
- `setIsOpen` は Promise を返さないで `await` は不要。
- `fetchData()` は非同期なので `await` を付け、一覧の取り直しが終わるのを待つ。

2. エラー 23502 (title に null が入る) の原因と修正

発生したエラー

```
POST .../rest/v1/study-record 400 (Bad Request)
code: "23502"
message: 'null value in column "title" of relation "study-record" violates not-null constraint'
details: "Failing row contains (16, null, null)."
```

23502 は PostgreSQL のエラーコードで、**NOT NULL 制約違反**を表します。「title 列は空にできないのに null が送られてきた」という意味です。

原因：送信ボタンから onSubmit が2回呼ばれていた

```
<form onSubmit={handleSubmit(onSubmit)}>
  ...
  <Button onClick={onSubmit} type="submit">登録</Button>
</form>
```

呼び出し経路	onSubmit に渡る値	結果
<code>onClick={onSubmit}</code>	クリックイベント (title も time も持たない)	title・time が <code>undefined</code> → null として送られ エラー
<code>type="submit" → handleSubmit(onSubmit)</code>	フォームの入力値	正常に登録

修正

```
<Button type="submit" bg="teal.600">登録</Button>
```

ポイント

- `<form>` 内の `type="submit"` ボタンを押すと、フォームの `onSubmit` が実行される (HTML 標準の動き)。
- React Hook Form の `handleSubmit` は「入力チェック → 通過したら入力値を渡して関数を実行」してくれる。送信は `handleSubmit` に任せ、ボタンに `onClick` は付けない。

3. コンソールの [vite] ログと、データ取得が2回走る理由

[vite] connecting... / [vite] connected.

画面が2回読み込まれているわけではありません。開発サーバー (Vite) との接続の「開始」と「完了」を表す1組のログです。ファイル保存時に画面を自動更新する **HMR (Hot Module Replacement)** のための接続で、`npm run dev` の開発中だけ表示されます。

Supabase へのリクエストが2回出る理由 : StrictMode

`src/main.tsx` でアプリ全体が `<StrictMode>` で囲まれています。

```
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <ChakraProvider value={system}>
      <App />
    </ChakraProvider>
  </StrictMode>,
)
```

StrictMode は開発中だけ、コンポーネントを「表示 → いったん外す → もう一度表示」と動かします。そのため `useEffect` が2回実行され、`select` のリクエストも2回出ます。

結論：バグではないので、このままでOK

- 後片付け漏れ（タイマーの解除忘れなど）を開発中に見つけるための、意図的な動き。
- 本番ビルド（`npm run build`）では1回だけ実行される。
- StrictMode は消さずに残しておくのがおすすめ。

4. 編集モードでの更新機能

修正前の問題点

#	問題	内容
1	更新関数が呼ばれていない	<code>onClick={() => onClickUpdate}</code> は関数を「返す」だけで「実行」していない（ <code>()</code> がない）。
2	フォームの入力がぶつかる	新規登録フォームと、行の数だけある編集フォームが、すべて同じ <code>useForm</code> で <code>title</code> / <code>time</code> を登録していた。同じ名前の入力が複数あり、どの値が送られるかわからない。
3	<code>update</code> に配列を渡していた	Supabase の <code>update</code> にはオブジェクト <code>{...}</code> を渡す（ <code>insert</code> は配列も可）。
4	エラーメッセージの文言	更新処理なのに「データ削除エラー」と出力していた。

新しい仕様（動きの流れ）

- 一覧の鉛筆アイコンを押すと `onClickEdit(record)` が実行される。
- 押された行を `editingRecord`（state）に入れ、`resetEdit` でフォームにその行の値をセットする。
- `editingRecord` が null でないので、編集モードが開く。
- 「更新する」（`type="submit"`）を押すと、`handleSubmitEdit` が入力チェックをする。
- チェックを通ると `onSubmitEdit` が、`editingRecord.id` の行を Supabase で更新する。
- 成功したら `setEditingRecord(null)` でモードを閉じ、`fetchData()` で一覧を取り直す。失敗したらモードは開いたまま。
- キャンセル・xボタン・背景クリックで閉じたときは、`onOpenChange` で `editingRecord` を null に戻す。

実装のポイント

① 編集用に別の `useForm` を用意する

```
const {
  register: registerEdit,
  handleSubmit: handleSubmitEdit,
  reset: resetEdit,
  formState: { errors: editErrors },
} = useForm<{ title: string, time: number }>();
```

分割代入で名前を付け替え（`register: registerEdit`）、新規登録フォームの `register` などとぶつからないようにしています。

② 編集中のレコードを state で持ち、reset で値を入れる

```
const [editingRecord, setEditingRecord] = useState<Recode | null>(null);

const onClickEdit = (record: Recode) => {
  setEditingRecord(record);
  resetEdit({ title: record.title, time: record.time });
};
```

React Hook Form では `defaultValue` を入力欄に直接書くより、`reset()` でフォーム全体の値を入れ直すほうが確実です。

③ 更新処理

```
const onSubmitEdit = async (data: { title: string, time: number }) => {
  if (!editingRecord) return;
  const { error } = await supabase
    .from('study-record')
    .update({ title: data.title, time: data.time }) // ← オブジェクトで渡す
    .eq('id', editingRecord.id);                // ← 更新する行を id で指定
  if (error) {
    console.error("データ更新エラー:", error);
    return;
  }
  setEditingRecord(null); // 成功時のみ閉じる
  await fetchData();
};
```

④ 編集モダールは map の外に1つだけ置く

```
<Dialog.Root
  open={editingRecord !== null}
  onOpenChange={(e) => { if (!e.open) setEditingRecord(null) }}
>
  ...
  <form onSubmit={handleSubmitEdit(onSubmitEdit)}>
    <Input id='edit-text' {...registerEdit("title", { required: "学習内容は必須です" })}>
  />
  ...
  <Button type="submit">更新する</Button>
</form>
</Dialog.Root>
```

行ごとにモダールを作ると、閉じている間も隠れた状態で画面に残り、同じ名前の入力が増えていきます。モダールを1つにして「どの行を編集するか」を state で切り替えるほうが、シンプルで不具合も起きにくい作りです。

鉛筆アイコンは、キーボードでも押せるように削除ボタンと同じく `<button>` で囲みました。入力欄の `id` は、新規登録フォームとかぶらないよう `edit-text` / `edit-time` にしています (HTML の `id` はページ内で一意にする)。

5. 残っている課題

- **新規登録フォームの型エラー**：新規登録の `useForm()` に型が付いていないため、`{errors.title.message}` で TypeScript のエラーが出ています。編集用と同じく `useForm<{title: string, time: number }>()` と型を付けると解消します。
- **isLoading が使われていない**：`setIsLoading(true)` しかしておらず、`false` に戻す処理も、画面で使う場所ありません。ローディング表示を作るなら、取得完了時に `setIsLoading(false)` を追加します。
- **使っていない state・関数**：`inputText`・`inputTime`・`open`・`totalTime`・`onChengeInputText` などが未使用です（React Hook Form に移行したため）。不要なら削除できます。
- **動作確認**：編集機能はブラウザでの動作確認がまだです。

6. 参考サイト

テーマ	参考サイト
Supabase：データの登録 (insert) 戻り値の error の扱い	https://supabase.com/docs/reference/javascript/insert
Supabase：データの更新 (update) update + eq の書き方	https://supabase.com/docs/reference/javascript/update
PostgreSQL のエラーコード一覧 23502 = not_null_violation	https://www.postgresql.org/docs/current/errcodes-appendix.html
React Hook Form：useForm	https://react-hook-form.com/docs/useform
React Hook Form：register	https://react-hook-form.com/docs/useform/register
React Hook Form：handleSubmit	https://react-hook-form.com/docs/useform/handlesubmit
React Hook Form：reset 編集時にフォームへ値を入れる	https://react-hook-form.com/docs/useform/reset
React：イベントへの応答 onClick={fn} と onClick={() => fn()} の違い	https://ja.react.dev/learn/responding-to-events
React：StrictMode 開発中に Effect が2回動く理由	https://ja.react.dev/reference/react/StrictMode
React：エフェクトを使って同期 を行う useEffect とデータ取得	https://ja.react.dev/learn/synchronizing-with-effects
Chakra UI：Dialog open / onOpenChange による開閉 制御	https://chakra-ui.com/docs/components/dialog
MDN：<button> 要素 type="submit" の動き	https://developer.mozilla.org/ja/docs/Web/HTML/Reference/Elements/button
Vite：機能（HMR） [vite] connected ログの正体	https://ja.vite.dev/guide/features